A Surrogate-based Approach for Faster Multi-objective Architectural Refactoring Optimization

J. Andres Diaz-Pace
ISISTAN-CONICET, *UNICEN University*
Tandil, Argentina
andres.diazpace@isistan.unicen.edu.ar
cturesDaniele Di Pompeo
SpencerLab, *University of L'Aquila*
L'Aquila, Italy
daniele.dipompeo@univaq.it
Antonela Tommasel
ISISTAN-CONICET, UNICEN University
Tandil, Argentina
Johannes Kepler University
Linz, Austria
antonela.tommasel@isistan.unicen.edu.ar

A Surrogate-based Approach for Faster Multi-objective Architectural Refactoring Optimization

***Abstract*—Software model optimization is a process that generates architecture alternatives aimed at improving quantifiable non-functional properties of software systems, such as performance and reliability. Multi-objective evolutionary algorithms are commonly used to explore the search space and help designers identify trade-offs among competing non-functional properties (e.g., through a Pareto front). However, such algorithms face efficiency challenges in complex software models, since evaluating the fitness (i.e., the quality) of each architecture requires analysis tools that become computationally expensive when repeatedly invoked during the search process. In this paper, we explore the construction of surrogate models based on machine learning techniques to approximate the outputs of these analysis tools at significantly lower computational cost, maintaining reasonable output accuracy. As the optimization engine generates new populations, the surrogate model is incrementally trained from a small sample of solver-evaluated architectures and used to predict the non-functional properties of the remaining instances of the population. To this end, we extended a multi-objective architecture optimization engine to integrate surrogate models and conducted experiments using the `CoCoME` and `Train Ticket` case-studies from the literature. Our results suggest that surrogate models are a promising approach toward scaling multi-objective architecture optimization to larger and more complex spaces of architectural models.**

***Index Terms*—Software Architecture, Search-Based Software Engineering, Multi-objective Optimization, Surrogate Models, Machine Learning**

I. Introduction

A software architecture is a high-level abstraction of a system that defines its internal structure in terms of components, connectors, properties and main functions. An architecture plays a crucial role in determining the blueprint of the software system and the non-functional properties that it should satisfy, such as maintainability, scalability, or performance, among others [1, 2]. Software architectures have been exploited to analyze non-functional properties of software systems [3, 4, 5]. These efforts normally rely on predefined analysis models (or solvers), such as Layered Queuing Networks (LQNs) for performance [6] or rule-based detection for performance antipatterns [7]. Beyond analysis, architectural refactoring has emerged as a complementary activity aimed at systematically modifying the structure of an architecture (*e.g.*, by redistributing components, introducing replicas, or restructuring communication paths) to improve quality attributes [8, 9, 1]. For example, Zhao et al. [9] analyzed the energy efficiency of microservices varying the granularity of the architectural decomposition. Ni et al. [1] improved performance of software architecture through rule-based refactoring.

When competing properties must be optimized together, multi-objective techniques—such as genetic algorithms [2, 10]—are well suited to this task, particularly when the properties can be expressed as numerical quantities. However, non-functional properties might be challenging to optimize due to the complexity of the software architectures, the computational cost of running the analysis tools, and the large search space of possible configurations [3, 1]. A common strategy to mitigate this cost is to impose evaluation or time budgets on the optimization process [11, 12]. Although effective at limiting runtime, such constraints may hinder the search from fully exploring the design space, potentially affecting the quality of the resulting architectural alternatives.

In this work, we address the high computational cost of evaluating the objectives via solvers and the need for extensive search efforts, which hinder multi-objective architectural optimization in practice. To this end, we propose a novel *approach to optimize software architectures using surrogate models to estimate non-functional properties of architectures*, and thus, approximate the behavior of the solvers with cheaper models. In particular, our approach leverages regression techniques to build surrogate models that predict non-functional properties based on features extracted from the architectural models handled by the optimization engine.

We evaluate our approach on two case studies in the literature: *CoCoME* [13] and *TrainTicket* [14, 15]. Furthermore, we compare the surrogate results with those achieved by a state-of-the-art search-based optimization technique [16]. Through this investigation, we aim to answer two research questions to shed light on the potential of surrogate models to improve the efficiency of multi-objective architectural optimization.

- **RQ1:** Can surrogate models reduce the overall execution time of multi-objective architectural refactoring optimization?
- **RQ2:** To what extent do surrogate models affect the quality of the Pareto front obtained through multi-objective architectural refactoring optimization?
- **RQ3:** Are there differences in the structural characteristics of the architecture candidates generated by a surrogate-assisted approach and those generated by a standard search-based optimization engine?

Evaluation showed that surrogate models can significantly reduce the evaluation time of non-functional properties during the optimization process, leading to faster convergence. The tradeoff here is less variability of solutions in the exploration of the design space, as outlined by quality indicators employed to evaluate quality of Pareto fronts. The results also highlighted that surrogate models seem to be case-dependent, and the complexity of the architecture models involved has an impact on their effectiveness.

II. Related Work

Although multi-objective optimization techniques, such as evolutionary algorithms, can yield good-quality and diverse solutions in different engineering domains, one of their main drawbacks is their execution time until reaching convergence.

Along this line, the definition of stopping criteria for the optimization process is a common strategy explored in the literature. However, this is often challenging in real-world settings, where formal representations or convergence are difficult to define [17]. Several approaches have relied on quality indicators [18, 19] or statistical tests over convergence metrics [20, 21]. Other works have focused on time- or cost-constrained search. For example, Arcuri and Fraser [22, 12] studied showed that fixed time budgets influence search-based test generation, while Luong et al. [23] proposed a cost-aware Bayesian optimization method that explicitly incorporates the evaluation cost of candidate solutions. In the context of software architecture optimization, Diaz-Pace et al. [11] examined how time budgets affect multi-objective refactoring search. Their findings indicate that, although budget-aware search reduces overall evaluation time, it may also lead to suboptimal architectural trade-offs depending on the search strategy, highlighting the sensitivity of architecture-level optimization to budget constraints. These contributions reflect a broader need to manage the computational expense of repeatedly invoking objective functions, an issue that becomes particularly relevant in architecture optimization, where non-functional analyses can rely on complex solver chains.

Surrogate models offer a complementary strategy by learning fast approximations of expensive evaluation functions. Widely used in engineering domains such as aerodynamics, structural optimization and environmental modeling, surrogates reduce the number of costly simulations required during search by predicting objective values at negligible computational cost. Their effectiveness has been demonstrated in surrogate-assisted evolutionary algorithms [24], where predictions replace a portion of fitness evaluations to accelerate convergence on expensive multi-objective problems.

Recent research has examined surrogate-based optimization specifically for system architecture problems, which are characterized by hierarchical, mixed-discrete, and multi-objective design spaces. Prior work has proposed Bayesian optimization with specialized structure [25], as well as methods that explicitly model hidden constraints such as solver failures or infeasible geometries to improve search performance [26]. Also, Bussemaker et al. [27] evaluated how well surrogate-assisted optimizers approximate the true Pareto front under fixed evaluation budgets, how efficiently they progress under different convergence criteria, and how robust they remain in the presence of hidden constraints. Using an analytical benchmark that reflects properties of architecting tasks, the authors showed that surrogate-based methods can reduce evaluation costs without sacrificing solution quality. These findings reinforce the suitability of surrogate modeling for (software architecture) design problems where each evaluation is costly and embedded in a large combinatorial search space.

Despite their success in other domains, surrogates remain comparatively underexplored in software engineering. Within model-based software architecture, the *Palladio Component Model* is a common Architecture Description Language (ADL) [28] that supports automated analysis and optimization of non-functional properties. Two examples are *PerOpteryx* [4] and *SQuAT* [5], which use Layered Queuing Networks to evaluate sets of architecture candidates. These evaluations are computationally expensive when invoked during multiple cycles of a multi-objective optimization.

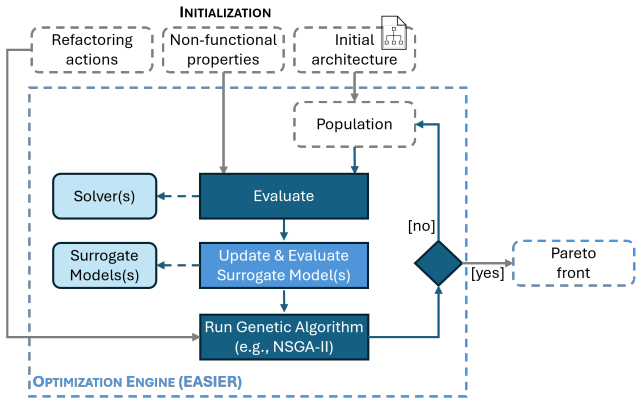


Fig. 1: Workflow of the proposed approach integrating surrogate models into a genetic algorithm engine for architectural refactoring optimization.

Only a few works have attempted to introduce surrogate models in architecture optimization. The closest one is [29], which trained surrogates to approximate performance and modifiability metrics in tree-based architectural search. While the results demonstrate the feasibility of predicting quality-attribute values from architectural features, the design of the approach had practical limitations for "live" optimization scenarios. For instance, the surrogates were trained offline on precomputed datasets and had no influence on the optimization process. Furthermore, the feature encoding schema required the solution candidates to be known in advance.

In contrast to these earlier efforts, this work integrates surrogate models directly into the evolutionary architecture-refactoring algorithm. Our surrogates are trained incrementally on solver evaluations generated during search, and their predictions actively guide exploration by replacing a portion of costly solver invocations. This design allows the optimization to scale to larger design spaces while maintaining the quality of the solutions being explored.

III. Approach

The multi-objective optimization process establishes a mapping between candidate architecture solutions and a multi-valued response surface, where each point represents the values of the non-functional objectives of interest. From a data-driven perspective, this process generates new pairs of architecture solutions and objective values, progressively forming a dataset that becomes the foundation for surrogate modeling.

Our work builds on the architectural refactoring optimization framework in [30], where a genetic algorithm (GA) iter-

atively transforms architecture models using predefined refactoring actions and evaluates each candidate through solver-based analyzes of selected non-functional properties. We extend this approach by integrating surrogate models (SMs) into the evaluation loop, as illustrated in Figure 1. The workflow begins with the initialization of a population of candidate solutions, each representing an architectural alternative of the software model. Through standard GA operators (crossover and mutation), the engine generates new offspring at each iteration. The fitness of each candidate can then be computed either by invoking the solvers, or, when appropriate, by querying an SM. When an SM is used, objective values are predicted from features extracted from the candidate architecture.

In each GA iteration, only a subset of candidates is evaluated using the solvers to obtain ground-truth objective values, while the remaining candidates are evaluated by the SMs. An initial set of surrogates is trained offline and injected into the optimization engine before the search begins. To provide greater flexibility, we maintain one SM per objective, allowing each surrogate to evolve independently based on the distribution and difficulty of its corresponding objective.

As surrogate predictions inevitably introduce approximation errors, and because the GA may explore regions of the design space that were not represented in the initial training data, the SMs are incrementally updated through the search. Their training sets are augmented at each iteration with samples from the most recent solver evaluations, following the incremental learning strategy proposed in [29]. This continuous retraining helps maintain the relevance and accuracy of the surrogates as the GA exploration progresses.

A. Objectives

We consider four objectives (power consumption, architectural changes, cost, and reliability), which capture essential trade-offs in modern software architecture design. Power and cost relate directly to the sustainability and operational efficiency of a system, while architectural change effort and reliability reflect its long-term maintainability and robustness. Optimizing these properties together supports the development of systems that are efficient to operate, resilient in use, and adaptable over time.

a) Power consumption: To estimate server power consumption when only maximum power usage is known, Xu and Jian [31] proposed to decompose power into idle and active CPU components, acknowledging the CPU as the dominant contributor [32, 33]. We adopt this approach and compute system power by aggregating power consumption across all deployment nodes (servers):

$$Power = \sum_{n \in N} \left[k \left(1 - U_n\right) power_{max}(n) + U_n \, power_{max}(n) \right] \quad (1)$$

where n denotes a node, $power_{max}(n)$ is its maximum power consumption at full utilization, k is a scaling factor that reduces $power_{max}$ during idle periods, and U_n represents the utilization of node n.

b) Architectural changes: This objective quantifies the effort required to apply refactoring actions, referred to as *Architectural changes*, and is computed as:

$$Arch.changes = \sum_{a=1}^{l} C_{base}(a) \cdot C_{arch}(a, e) \quad (2)$$

where $C_{base}(a)$ represents the base cost associated with refactoring action type a, and $C_{arch}(a, e)$ denotes the architectural cost determined by the target element e being modified. The architectural cost $C_{arch}(a, e)$ depends on the characteristics of the refactoring target element. Highly interconnected elements incur greater architectural costs due to their wider impact, while elements with fewer dependencies require less effort to modify as their changes have more localized effects on the architecture.

c) Cost: The deployment cost captures the financial expense of hosting a software system on cloud infrastructure, determined by the number and type of hardware nodes employed[1]. It is calculated as the sum of the hourly costs of all nodes in the system:

$$Cost = \sum_{n \in N} cost(n) \quad (3)$$

d) Reliability: We adopt the model of [34], which defines the mean failure probability θ_S of a system S as:

$$\theta_S = 1 - \sum_{j=1}^{K} p_j \left(\prod_{i=1}^{N} (1 - \theta_i)^{InvNr_{ij}} \cdot \prod_{l=1}^{L} (1 - \psi_l)^{MsgSize(l,j)} \right) \quad (4)$$

This formulation combines component (θ_i) and communication links (ψ_l) failure probabilities with execution probabilities (p_j) to derive the system reliability on demand (θ_S), representing the likelihood that the system will not fail when its scenarios are invoked.

B. Refactoring Actions

The refactoring catalog employed in this study comprises actions designed to enhance performance as well as other non-functional properties of a software system. A feasibility engine presented in [3] ensures the validity of the sequences of refactoring actions. We focus on four refactoring actions:

a) `redo` *- Redeploy Existing Component:* This action relocates a selected component to a newly created node, modifying the deployment view. The newly created node establishes connections with all nodes directly linked to the original deployment node, thereby redistributing components to optimize the system architecture.

b) `move` *- Relocate Operation to Existing Component:* This action transfers a specific operation to an existing target component. It modifies each scenario involving the operation, improving system adaptability and efficiency.

c) `clone` *- Clone Node:* This action creates a replica of an existing node along with all its deployed artifacts and connections, enhancing system redundancy and fault tolerance.

d) `motn` *- Move Operation to New Component on New Node:* This action moves an operation to a newly created component deployed on a new node, requiring coordination between dynamic and deployment views. The new component is added to the dynamic view, while the new node, artifact, and

[1]Amazon EC2 Spot Price History: https://zenodo.org/doi/10.5281/zenodo.5880792

associated links are created in the deployment view. Messages involving the relocated operation are redirected to ensure seamless integration.

C. Deployment Nodes for Architectural Models

TABLE I: Deployment Nodes and Descriptions. Each Amazon EC2 instance type has been labeled w.r.t. *Power Consumption* in Watt, and *Cost* in USD/h. We used a 3-Point Likert Scale: low (L), medium (M), and high (H) to map the characteristics of such a deployment node.

Instance Type	Description	Power Consumption	Cost
`d2.2xlarge`	Designed for resource-intensive workloads like scientific simulations.	83.4 (H)	0.46 (H)
`m6i.xlarge`	Offers good performance for demanding web applications.	32.4 (M)	0.13 (M)
`t2.medium`	Well-balanced for a wide range of general-purpose workloads.	14.1 (M)	0.03 (L)
`t2.micro`	Best suited for non-resource-intensive tasks like simple web hosting.	6.40 (L)	0.004 (L)
`m5ad.xlarge`	Suitable for tasks where a balance between energy efficiency and operational cost is desired.	29.9 (M)	0.25 (M)

To model deployment nodes, we selected a subset of representative Amazon EC2 instances[2], namely *d2.2xlarge*, *m5ad.xlarge*, *t2.medium*, *t2.micro* (Table I), chosen to capture diverse cost-performance trade-offs. Instance costs were taken from the Amazon EC2 Price History dataset[3].

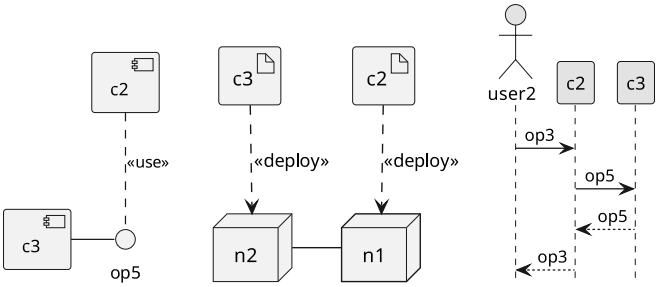
D. Modeling Assumption

In this study, we utilize UML to model the software architectures under optimization. Specifically, we utilize component diagrams to represent static relationships among software components, sequence diagrams to capture dynamic system behavior (*e.g.*, sequences of operations), and deployment diagrams to model the hardware architecture, as depicted in Figure 2.

Standard UML does not natively support performance modeling, for this reason we leverage established UML profiles for performance analysis [35, 30]. Furthermore, we adopt Layered Queueing Networks (LQN) [6], a stochastic model suitable for forecasting software system performance.

E. Feature Encoding of Architectural Models

Our SMs require a representation that maps each architecture of a population to a format (*i.e.*, features) suitable for regression. Since each architecture is defined by a sequence of refactoring actions applied to the initial architecture, this representation cannot be processed directly by the SMs. Instead, we construct a tabular, numeric representation in which each candidate is converted into a fixed-length feature vector describing its refactoring and architectural characteristics. This representation (or encoding) is defined once at the onset and remains stable across all subsequent populations, considering that refactoring actions might introduce new nodes or components in some candidates.



(a) Component (b) Deployment (c) Sequence

Fig. 2: UML diagrams for our architectures.

We focus on the four refactoring actions mentioned above (`redo`, `move`, `motn` and `clone`). To encode the sequence structure in a way that is both compact and expressive, we borrow ideas from natural language processing [36, 37] and biomolecular sequence analysis [38]. In those fields, sequences (either words in a sentence or amino acids in a protein) are often described not only by the individual elements that occur, but also by the short-range patterns in which they appear. In this context, our encoding includes *action unigrams*, which count how many times each refactoring action occurs, and *action bigrams*, which count how often one action immediately follows another. While unigrams reflect the overall distribution of refactoring types, bigrams capture local dependencies that often carry architectural meaning. For example, a `clone` followed immediately by a `motn` suggests a distinct refactoring pattern that contributes to fault tolerance and load distribution, compared to the same actions appearing far apart in the sequence. By incorporating short-range co-occurrence patterns, we obtain a lightweight but informative description of how refactoring actions interact to lead to specific candidates.

Figure 3 shows how a sequence with four refactoring actions gets mapped to a feature vector. Features are grouped into different categories, namely: elements (nodes, components) created, action counts (unigrams), bigrams, positional action counts, device families, and device sizes, among others.

To retain information about the temporal structure of the transformation process, we additionally encode which actions appear at each position of the sequence. As some refactoring actions can introduce new architectural elements, the encoding also includes indicators quantifying the structural expansion of the architecture, such as the number of newly created components and nodes. For instance, having more components or nodes often correlates with horizontal scaling, functional decomposition, increased architectural complexity or potential

[2]Amazon EC2 Instances Carbon Footprint Estimator: https://docs.google.com/spreadsheets/d/1DqYgQnEDLQVQm5acMAhLgHLD8xXCG9BIrk-_Nv6jF3k [Accessed: 2025-12-06]

[3]Amazon EC2 Spot Price History: https://zenodo.org/doi/10.5281/zenodo.5880792

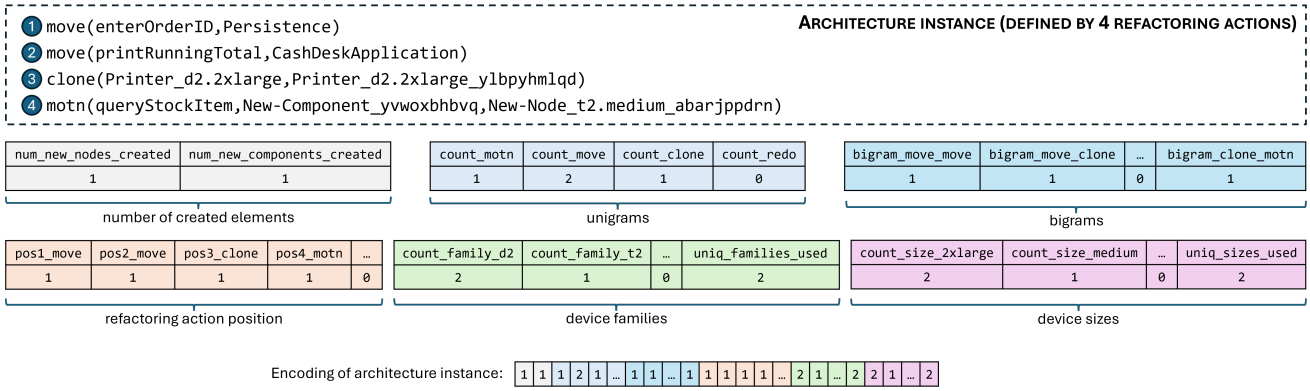


Fig. 3: Encoding of a sequence of refactoring actions into feature vector, with different feature categories.

points of failure, among other aspects.

Another challenge is that node identifiers in the architectural model often embed hardware information in non-uniform strings (*e.g.*, `StoreServer_d2.2xlarge`). To retain this contextual information without relying on raw identifiers, we normalized node names by extracting two stable tokens: the hardware *family* (*e.g.*, `t2`, `m6i`, `d2`) and the *size* class (*e.g.*, `medium`, `xlarge`). These tokens are mapped to a finite vocabulary determined at initialization, and their occurrences are tracked across the refactoring actions.

The proposed encoding provides a fixed-size, interpretable numerical vector for each architecture candidate. As the vocabulary and feature dimensionality are fixed for any population throughout the optimization, the encoding supports incremental re-training without schema drift between populations.

F. Incremental Regression Models

Once the architecture candidates are encoded, their non-functional objective values are seen as regression targets for the SMs. Regression is a type of supervised learning in which the goal is to predict continuous values based on input data. Since we deal with multiple objectives, multi-output regression is applied [39]. In our case, the model internally creates a single regressor per objective.

The incremental learning strategy (see Figure 1) is intended to reflect the dynamic nature of an optimization process in which populations are generated in batches as the GA proceeds. For the SMs, an initial regression model is learned and then progressively updated with a fraction of the new data available from each population. Gradient boosting techniques are appealing in this scenario, because the underlying estimators (tree ensembles) can be updated with newly-trained estimators for incoming data. In this way, we can adjust the SM with new architecture candidates while still retaining patterns extracted from previous instances.

IV. Evaluation

We evaluated our approach by formulating a set of research questions that guided both the design and analysis of our experiments. These research questions allow us to clarify how surrogate models can support the efficiency of multi-objective architectural refactoring optimization.

- **RQ1:** Can surrogate models reduce the overall execution time of multi-objective architectural refactoring optimization?

  This question investigates whether integrating surrogate models into the optimization process yields meaningful reductions in execution time compared to exclusively relying on solver-based analyses.
- **RQ2:** To what extent do surrogate models affect the quality of the Pareto front obtained through multi-objective architectural refactoring optimization?

  This question examines the effect of surrogate models on the quality of the architectural solutions produced by the optimizer, using indicators such as hypervolume and generational distance to determine how closely the obtained front approximates the true one.
- **RQ3:** Are there differences in the structural characteristics of the architecture candidates generated by a surrogate-assisted approach and those generated by a standard search-based optimization engine?

  Unlike the previous question that looks at the objective space, this question focused on the architectural models being explored during the optimization process, and how surrogate models shape the use of refactoring actions for the architecture candidates.

A. Experimental Setup

We compared a standard multi-objective architecture optimization setup using *NSGA-II* as the underlying GA [16]) with our surrogate-assisted approach, in which regression models approximate solver evaluations. Surrogates were implemented using*XGBoost*, which supports incremental updates, aligned with the iterative nature of the GA. For each iteration, 50% of the candidate architecture instances were evaluated with the solver, while the remaining instances were assigned objective values predicted with the SMs.

Experiments were conducted on two well-known case studies from the software architecture optimization literature. These systems were chosen because they exhibit different characteristics, allowing us to test our approach under varied conditions. We evaluated performance along two dimensions: computational time (**RQ1**), and quality indicators for the resulting Pareto front (**RQ2**).

The GA configuration followed common settings from the literature [9, 3, 1]: a population of 12 architecture candidates, chromosomes (*i.e.*, a sequence of refactoring actions) of length 4, and 102 iterations per run. Each configuration was executed 31 times on each system to mitigate variability[4].

All experiments ran on a cluster of three Dell PowerEdge C6525 servers, each equipped with two AMD EPYC 7282 2.80GHz CPUs and 512 GiB of RAM.[5]

B. Case Studies

We evaluated our approach on two systems commonly used in software optimization research: *CoCoME* [13] and *TrainTicket* [14, 15]. Both case studies include objectives for power consumption, architectural changes, cost, and reliability.

CoCoME (*CCM*) is a reference system for non-functional model-based analyses. It describes a trading system consisting of several stores, each containing one or more cash desks [13]. The system captures typical operational scenarios such as scanning products, processing payments, and managing stock replenishment.

TrainTicket (*TTBS*) is a Web-based booking application built on a microservices architecture [14, 15]. The system comprises 40 microservices, supporting user interactions such as searching trips, booking tickets, and retrieving travel information.

C. Metrics

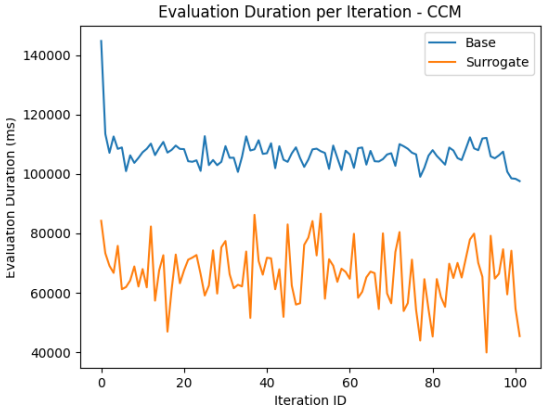
To evaluate the quality of the Pareto fronts produced by the optimization approaches, we rely on standard multi-objective quality indicators used in the literature. In particular, we employ Hypervolume (HV) [40, 41], Inverveted Generational Distance (IGD+) [42], Inverted Generational Distance (IGD+), and Epsilon (ϵ). These indicators were selected due to their prevalence in prior literature and their complementary nature, as discussed by Li and Yao [43]. For the indicators requiring a reference front, we constructed it by merging the non-dominant solutions from all runs. For `HV`, the reference point was defined as the worst value observed per objective across all executions.

Hypervolume (HV): HV measures the portion of the objective space dominated by the computed Pareto front with respect to a reference front ($referencePF$). Values range from 0 (fully dominated) to 1 (ideal, non-dominated). Higher values indicate better convergence and coverage.

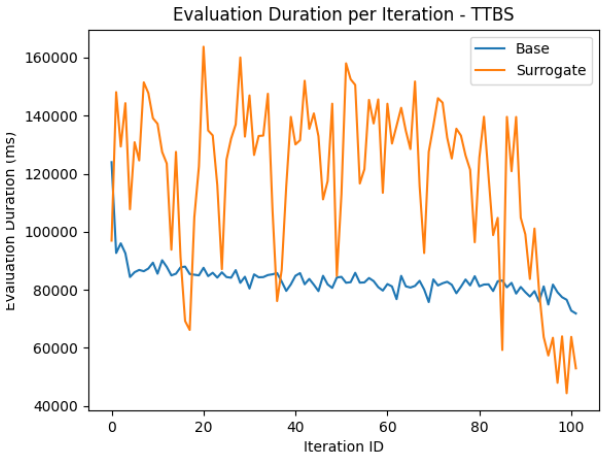
Generational Distance (GD+): GD+ quantifies the average distance from each solution in the computed front to the nearest solution in the reference front. Lower GD+ values indicate that the computed solutions are closer to the true Pareto front, with 0 representing perfect correspondence.

[4]These parameters reflect typical choices in architecture optimization studies, considering that each population member represents a full software architecture model and each evaluation involves running computationally intensive analysis tools.

[5]The replication package is available at https://doi.org/10.5281/zenodo.17846462



(a) *CCM*.



(b) *TTBS*.

Fig. 4: Execution time comparison between the standard GA and the surrogate-based GA for both case studies.

Inverted Generational Distance (IGD+): IGD+ measures, for each solution in the reference front, the distance to the closest dominating or non-dominated solutions in the computed front. This makes IGD+ more robust in capturing both convergence and diversity [42]. Lower values indicate higher-quality Pareto fronts.

Epsilon (ϵ): The ϵ indicator measures the maximum distance between two solution sets. For $\epsilon(computedPF, referencePF) \leq 0$, the computed front weakly dominates the reference front. Lower values indicate higher quality.

V. Results

A. RQ1: Reductions in overall execution time

Figure 4a reports the execution-time comparison between the standard GA and the surrogate-based GA for *CCM*. The surrogate-based GA achieved a substantial speedup across all iterations, reducing overall execution time by $\approx 38\%$. This reduction is primarily attributed to the reduced number of solver invocations, as SMs provide inexpensive approximations for half of the population in each iteration. The time required

to encode architecture instances was negligible (in both case studies), given the relatively small population size.

For *TTBS*, the execution-time comparison is shown in Figure 4b. In contrast to *CCM*, the surrogate-assisted GA was initially slower than the standard GA, becoming competitive only in the final iterations. This behaviour suggests that, for *TTBS*, the overhead of repeatedly retraining surrogate models can outweigh the savings obtained by reducing solver calls, specially during early generations. The late-stage improvements seem to indicate that surrogate retraining eventually stabilizes, enabling potential speedups once the models become more accurate and require fewer adjustments. Further investigation is needed to understand this effect, and, if necessary, constrain the retraining cost through budgeted or adaptive update strategies.

Summary: The surrogate-assisted GA yielded substantial execution-time reductions for *CCM*, achieving a speedup of $\approx 38\%$. However, this benefit did not generalize to *TTBS*, where surrogate overhead dominated and led to slower execution overall. These results indicate that the runtime benefits of surrogate models are context-dependent, influenced by factors such as system complexity, the stability of surrogate retraining, and the relative cost of solver evaluations.

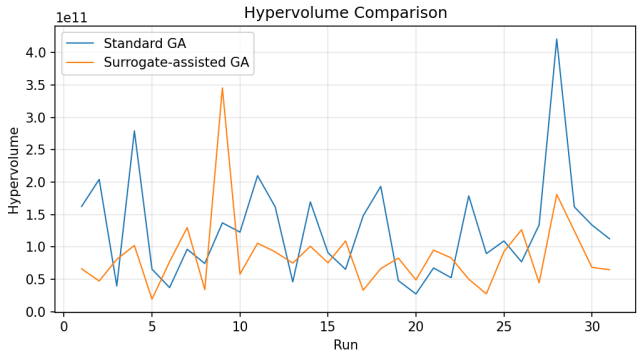
B. RQ2: Quality of the computed Pareto front

To address **RQ2**, we compared the Pareto fronts produced by the standard GA and the surrogate-assisted GA using four quality indicators described in Section IV-C.

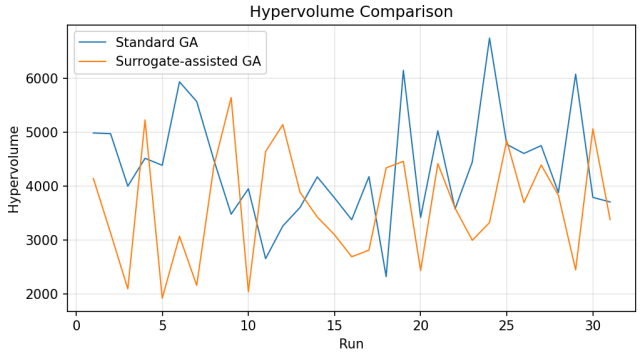
Across both case studies, the standard GA produced higher-quality Pareto fronts. `HV` (Figure 5a and Figure 5b) results generally favoured the standard GA (differences were less noticeable for *TTBS*), indicating that surrogate-assisted optimization tended to explore a narrower portion of the objective space. While the surrogate-based approach occasionally achieved comparable, and in isolated cases even higher `HV` values, the overall trend suggests reduced coverage of the solution space. This behaviour is consistent with the effects of surrogate approximation errors, which may steer the GA away from promising (but insufficiently represented) regions of the search space.

For `GD+` (Figure 6a and Figure 6b), both approaches showed similar convergence towards the reference front, suggesting that surrogate errors did not compromise proximity to optimal solutions. However, `IGD+` (Figure 7a and Figure 7b) revealed that the standard GA generally maintained better coverage and diversity (in particular for *TTBS*), while the surrogate-assisted GA tended to produce fronts with lower spread. This again aligns with the hypothesis that prediction errors constrain exploration.

For the `Epsilon` indicator, the results differed across case studies. In *CCM* (Figure 8a), the standard GA consistently achieved lower `Epsilon` values, indicating solutions that more closely dominated or approximated the reference front. In contrast, for *TTBS* (Figure 8b), both approaches exhibited very similar `Epsilon` values. This suggests that, while surrogate assistance can lag behind in some settings, it can also deliver Pareto fronts that are broadly comparable to the standard GA in others.



(a) *CCM*.



(b) *TTBS*.

Fig. 5: HV comparison between Standard GA and Surrogate-assisted GA quality for both case studies.

The statistical analysis (Wilcoxon signed-rank tests, Table III) confirms these trends. For *CCM*, `HV` and `Epsilon` showed statistically significant differences favoring the standard GA, with moderate effect sizes. `IGD+` showed a marginal effect in the same direction, while `GD+` exhibited no significant differences. Overall, these findings highlight that Standard GA provides noticeable improvements on several key indicators, particularly those with stronger effect sizes and lower p-values. For *TTBS*, none of the differences reached statistical significance at the 0.05 level. However, all indicators displayed a small-to-moderate effect size trend favoring the standard GA. Epsilon differences were negligible. Overall, while the Standard GA performed better on average across all metrics, the observed differences were not statistically significant and should therefore be interpreted with caution.

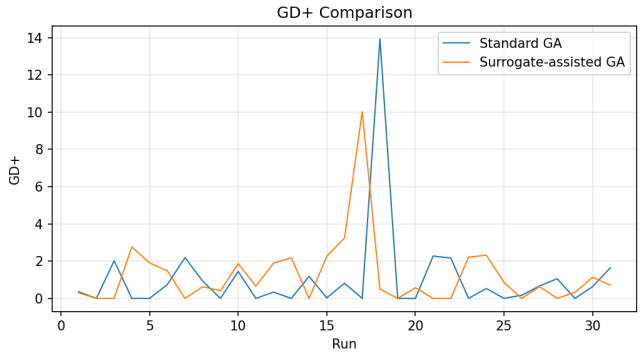
Summary: Overall, the standard GA achieved higher performance across the evaluated indicators, though the magnitude of these differences varied by indicator and case study. The surrogate-assisted GA, while not matching the solution quality of the standard GA, still produced reasonable and competitive Pareto fronts, demonstrating that surrogate-based guidance can maintain acceptable solution quality despite relying on approximate evaluations. The observed gaps are likely linked to prediction inaccuracies that may reduce exploration diversity or steer the search away from less familiar regions of the design space. Importantly, these findings highlight a fundamental trade-off, surrogate models can offer meaningful

Case Study	Mean		Min		Max		Q3	
	Std	S-based	Std	S-based	Std	S-based	Std	S-based
CCM	106764.32	**66290.69**	97598.00	**39918.00**	144763.00	**86640.00**	108511.75	**72633.00**
TTBS	**83364.08**	118891.79	71842.00	**44361.00**	**123964.00**	163738.00	**85161.75**	138641.50

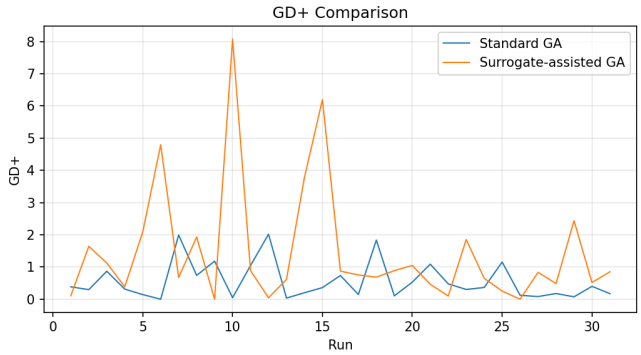
TABLE II: Execution time (ms) statistics for both case studies and algorithms. *Std* and *S-based* stand for standard GA and surrogate-based GA, respectively. The best values for each statistic (the lower, the better) are in bold.

Quality Indicator	stat		$p-value$		d_z	
	CCM	TTBS	CCM	TTBS	CCM	TTBS
HV	114.0000	132.0000	7.56e-03	2.21e-02	0.4484	0.4431
GD+	167.0000	144.0000	2.75e-01	4.14e-02	0.0549	0.4168
IGD+	148.0000	173.0000	5.02e-02	1.46e-01	0.3281	0.3168
Epsilon	121.0000	225.0000	1.17e-02	6.64e-01	0.5134	0.1718

TABLE III: Statistical comparison between Standard GA and Surrogate-assisted GA across multiple quality indicators for both case studies. The Wilcoxon signed-rank test (`W(Std, S-assisted)`) was used, lower *p-values* indicate statistically significant differences, and d_z denotes effect size. Standard GA outperformed Surrogate-assisted GA in all cases.



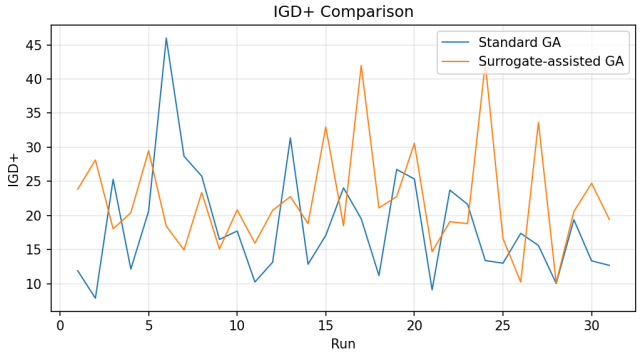
(a) *CCM*.



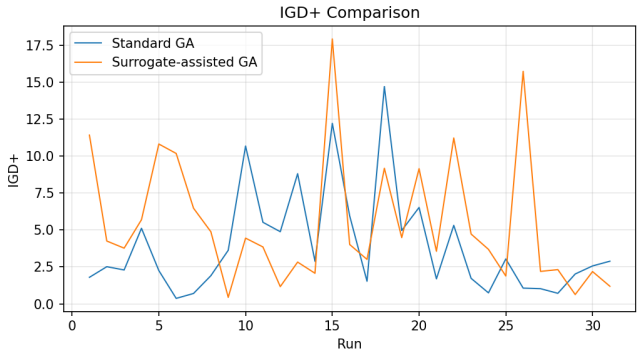
(b) *TTBS*.

Fig. 6: GD+ comparison between Standard GA and Surrogate-assisted GA for both case studies.

execution-time reductions (RQ1), but may introduce quality limitations (RQ2). This suggests that surrogate-assisted optimization remains promising (especially under constrained computational budgets) while also motivating further refinement of surrogate accuracy and integration strategies.



(a) *CCM*.



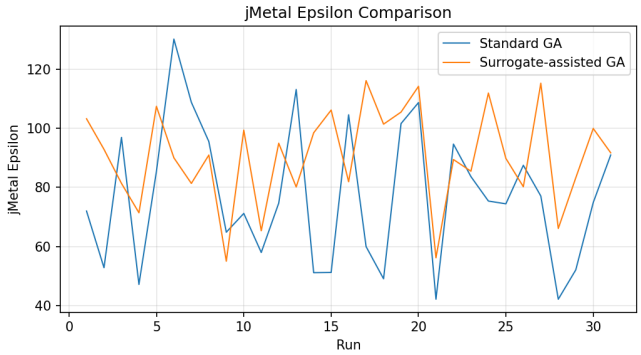
(b) *TTBS*.

Fig. 7: IGD+ comparison between Standard GA and Surrogate-assisted GA for both case studies.

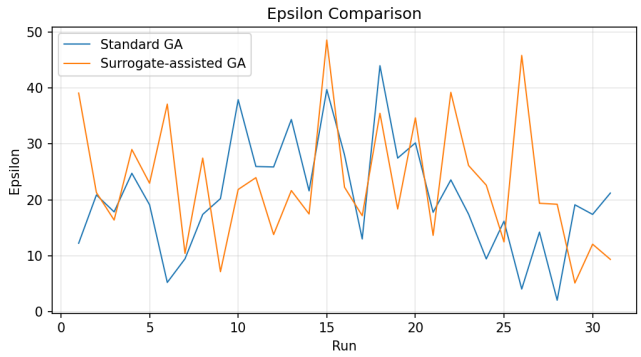
C. RQ3: Characterization of architectural models

To analyze possible model differences between the standard and surrogate-assisted GAs, we used the refactoring actions, described in Section III-B, as proxies for the software architecture, and computed their frequency at each position of the sequences explored in our experiments. The frequency for the four refactoring actions and positions of the sequences is summarized in Figures 9 and 10 for *CCM* and *TTBS*, respectively.

We considered all the sequences explored by standard and surrogate-assisted GAs, by concatenating the corresponding action types, and took their intersection. In *CCM* 57% of the sequences were shared by the two GAs, while in *TTBS* we obtained 67% shared sequences. This evidence suggests that both GAs have highlighted similar patterns in the usage of refactoring actions across the sequence positions, for both case studies, which answer **RQ3**. Moreover, analyzing the frequencies of the refactoring actions, emerges that the initial software architectures required a balanced distribution of

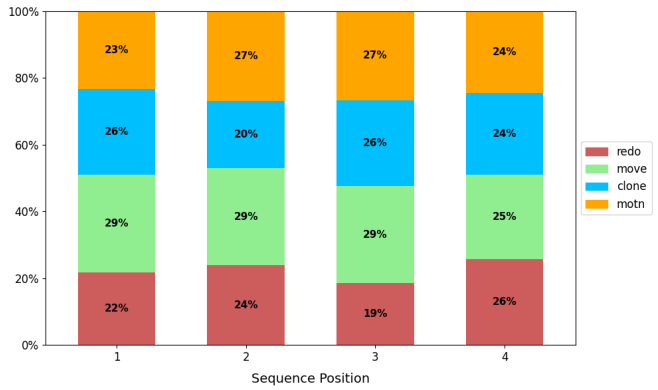


(a) *CCM*.

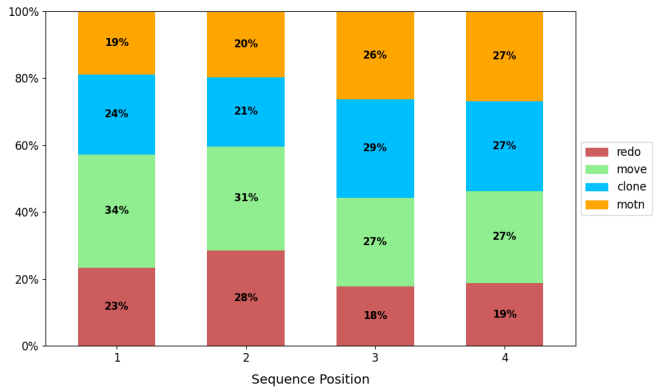


(b) *TTBS*.

Fig. 8: Epsilon comparison between Standard GA and Surrogate-assisted GA for both case studies.



(a) Standard GA.



(b) Surrogate-assisted GA.

Fig. 9: Profiles of refactoring actions per sequence position in *CCM*, considering all populations of architecture candidates.

components across the available deployment nodes to optimize the objectives, and both standard and surrogate-assisted GAs capture this need, as indicated by the frequent use of the *move* action. This insights might help architects to understand where the initial architecture needs to be improved.

Summary: Despite small variations, in both case studies, refactoring actions per sequence position between the standard and surrogate-assisted GAs have similar frequencies. Perhaps more importantly, a significant portion of the sequences explored by both GAs are shared, indicating that the surrogate models do not drastically alter the search space exploration in terms of architectural modifications.

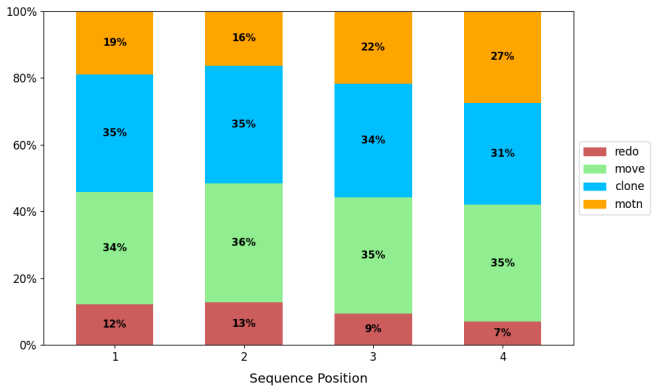
Lessons Learned: From our analysis, we observed that both the standard and surrogate-assisted GAs exhibited similar patterns in the usage of refactoring actions across sequence positions for both case studies, as shown in **RQ3**. Furthermore, standard and surrogate-assisted GAs can identify how the initial architecture needs to be improved, as indicated by the frequent use of the *move* action to achieve a balanced distribution of components. However, on the one hand, surrogate-assisted GA might suffer from lower quality in the Pareto front, as discussed in **RQ2**; on the other hand, surrogate models can speedup the execution time by reducing the invocations of computationally demanding solvers, as shown in **RQ1**.

VI. Threats to Validity

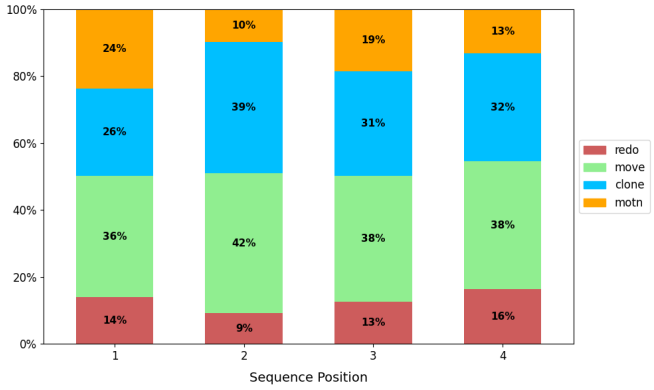
We identified several threats to internal, construct, and external, and conclusion validity in our experimental evaluation.

a) Construct validity: A threat arises from the lack of feature selection or feature importance analysis. As we included all features from our encoding schema without filtering them. Irrelevant or redundant features might have introduced noise into the regression models, while key features might have not been emphasized appropriately. In addition, we split the population instances into halves to decide which ones were tackled with the solvers and which ones were predicted with the SMs, but we did not try alternative sampling ratios. These factors could have affected the predictive accuracy of surrogates or their training times.

b) External validity: The generalizability of our results is limited by several experimental assumptions. First, our evaluation relies on architectural models expressed in a particular modeling notation. Although our UML notation is widely used in performance and reliability analysis, other modeling languages might allow different architectural structures, refactoring primitives, or analytic solvers. As a result, the behavior of SMs might vary when applied another setting. Second, our assessment is based on representative case studies from the literature, which differ in size, structure and complexity, but they might not reflect the diversity of real-world architectures (*e.g.*, industrial systems). A further threat arises from the limited refactoring space used by the optimization engine. Architecture candidates were generated using a predefined set of refactoring actions and a fixed sequence length, which stem



(a) Standard GA.



(b) Surrogate-assisted GA.

Fig. 10: Profiles of refactoring actions per sequence position in *TTBS*, considering all populations of architecture candidates.

from common architecture refactorings. However, a broader or domain-specific catalog of actions may produce different architectural configurations and non-functional values, and consequently, pose challenges for our SM. Finally, the datasets used are relatively small compared to typical machine learning benchmarks. Surrogate behavior, accuracy trends, and sampling effects may differ in larger or noisier datasets, especially those produced by long-running or industrial-scale optimization processes.

c) Internal validity: Parts of our machine learning pipeline involve inherent non-determinism. Although we controlled randomness through repeated runs and fixed seeds, different configurations and executions might lead to variations in training/test splits, sampling strategy or surrogate performance, potentially affecting our conclusions. Also, differences in the execution environment (e.g., hardware, engine configuration) might affect timing measurements.

d) Conclusion validity: We did not evaluate human factors such as how architects interpret or trust surrogate predictions, and the derived architecture candidates thereof, which might influence the practical adoption of surrogate-based optimization.

VII. Conclusion

In this work, we investigated the feasibility of using surrogate models to accelerate multi-objective quality-attribute optimization of architectural models. The central idea was to interleave expensive solver-based evaluations with fast, but potentially less accurate, surrogate predictions, thereby reducing computation time while preserving the effectiveness of the optimization process. Our study confirms that surrogate models can be integrated into the architecture optimization process with minimal overhead and that they offer meaningful efficiency gains in realistic settings, albeit with less variability on the Pareto fronts. Furthermore, the search space (*i.e.*, the structure of the architecture candidates) explored by the surrogate-assisted GA seemed slightly different from the one taken as the baseline.

We envision several directions for future work. Incorporating drift detection over incoming data batches would make it possible to selectively re-train the surrogates upon data distribution changes, thus improving the performance of the current approach. More advanced regression techniques (such as TabNet [44] or neural-based architectures) could capture richer feature interactions and potentially lead to better prediction quality for complex objectives. Exploring alternative encodings for architectural models and refactoring actions (*e.g.*, graph-based or embeddings techniques) may yield more informative feature spaces for surrogate modeling. Finally, the decision of which candidates to evaluate with solvers could be enhanced through adaptive sampling strategies that account for uncertainty, diversity, or predicted model reliability.

Taken together, these directions point toward a more general and adaptive surrogate-assisted optimization framework, one capable of tuning itself to the characteristics of the objectives, architecture space and search dynamics as the process unfolds. Such advancements hold promise to strengthen the applicability of surrogate models in large-scale, multi-objective optimization of architectural models.

References

[1] Y. Ni, X. Du, P. Ye, L. L. Minku, X. Yao, M. Harman, and R. Xiao, "Multi-objective software performance optimisation at the architecture level using randomised search rules," *Inf. Softw. Technol.*, vol. 135, p. 106565, 2021.

[2] I. Meedeniya, B. Buhnova, A. Aleti, and L. Grunske, *Architecture-Driven Reliability and Energy Optimization for Complex Embedded Systems*, ser. Lecture Notes in Computer Science, 2010, vol. 6093, p. 52–67.

[3] D. Arcelli, V. Cortellessa, M. D'Emidio, and D. Di Pompeo, "EASIER: an evolutionary approach for multi-objective software architecture refactoring," in *18th IEEE International Conference on Software Architecture, ICSA*, 2018, pp. 105–114.

[4] A. Busch, D. Fuchss, and A. Koziolek, "Peropteryx: Automated improvement of software architectures," in *IEEE International Conference on Software Architecture Companion, ICSA Companion 2019, Hamburg, Germany, March 25-26, 2019*, 2019, pp. 162–165. [Online]. Available: https://doi.org/10.1109/ICSA-C.2019.00036

[5] A. Rago, S. Vidal, J. A. Diaz-Pace, S. Frank, and A. van Hoorn, "Distributed quality-attribute optimization of software architectures," in *Proceedings of the 11th Brazilian*

Symposium on Software Components, Architectures, and Reuse, 2017, p. 1–10.

[6] G. Franks, T. Al-Omari, M. Woodside, O. Das, and S. Derisavi, "Enhanced modeling and solution of layered queueing networks," *IEEE Transactions on Software Engineering*, vol. 35, no. 2, pp. 148–161, 2008.

[7] V. Cortellessa and D. Di Pompeo, "Analyzing the sensitivity of multi-objective software architecture refactoring to configuration characteristics," *Information and Software Technology*, vol. 135, p. 106568, 2021.

[8] O. Zimmermann, "Architectural refactoring: A task-centric view on software evolution," *IEEE Softw.*, vol. 32, no. 2, pp. 26–29, 2015. [Online]. Available: https://doi.org/10.1109/MS.2015.37

[9] Y. Zhao, T. D. Matteis, and J. Bogner, "How does microservice granularity impact energy consumption and performance? A controlled experiment," in *22nd IEEE International Conference on Software Architecture, ICSA*, 2025, pp. 84–95.

[10] A. Martens, H. Koziolek, S. Becker, and R. Reussner, "Automatically improve software architecture models for performance, reliability, and cost using evolutionary algorithms," in *Proceedings of the first joint WOSP/SIPEW international conference on Performance engineering*, 2010, p. 105–116.

[11] J. A. Diaz-Pace, D. Di Pompeo, and M. Tucci, "On the role of search budgets in model-based software refactoring optimization," vol. 33, no. 1, p. 18.

[12] A. Arcuri and G. Fraser, "Parameter tuning or default values? an empirical investigation in search-based software engineering," *Empirical Software Engineering*, vol. 18, no. 3, p. 594–623, 2013.

[13] S. Herold, H. Klus, Y. Welsch, C. Deiters, A. Rausch, R. Reussner, K. Krogmann, H. Koziolek, R. Mirandola, B. Hummel, M. Meisinger, and C. Pfaller, *CoCoME - The Common Component Modeling Example*, ser. Lecture Notes in Computer Science, 2008, vol. 5153, p. 16–53.

[14] D. Di Pompeo, M. Tucci, A. Celi, and R. Eramo, "A microservice reference case study for design-runtime interaction in MDE," in *2nd International Workshop on Model-Driven Engineering for Design-Runtime Interaction in Complex Systems*, ser. CEUR Workshop Proceedings, vol. 2405, 2019, pp. 23–32.

[15] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *TSE*, vol. 47, no. 2, pp. 243–260, 2021.

[16] V. Cortellessa, D. Di Pompeo, and M. Tucci, "Exploring sustainable alternatives for the deployment of microservices architectures in the cloud," in *21st IEEE International Conference on Software Architecture, ICSA*, 2024, pp. 34–45.

[17] T. Wagner, H. Trautmann, and L. Martí, "A taxonomy of online stopping criteria for multi-objective evolutionary algorithms," in *Evolutionary Multi-Criterion Optimization - 6th International Conference*, R. H. C. Takahashi, K. Deb, E. F. Wanner, and S. Greco, Eds., vol. 6576, 2011, p. 16–30, dOI:.

[18] T. Wagner, H. Trautmann, and B. Naujoks, "Ocd: Online convergence detection for evolutionary multi-objective algorithms based on statistical testing," in *Evolutionary Multi-Criterion Optimization, 5th International Conference*, ser. Lecture Notes in Computer Science, M. Ehrgott, C. M. Fonseca, X. Gandibleux, J.-K. Hao, and M. Sevaux, Eds., vol. 5467, 2009, pp. 198–215.

[19] J. L. Guerrero, L. Marti, A. Berlanga, J. Garcia, and J. M. Molina, "Introducing a robust and efficient stopping criterion for moeas," in *IEEE Congress on Evolutionary Computation*, 2010, p. 1–8.

[20] H. Trautmann, U. Ligges, J. Mehnen, and M. Preuss, *A Convergence Criterion for Multiobjective Evolutionary Algorithms Based on Systematic Statistical Testing*, ser. Lecture Notes in Computer Science, 2008, vol. 5199, p. 825–836.

[21] L. Martí, J. Garcia, A. Berlanga, and J. M. Molina, "An approach to stopping criteria for multi-objective optimization evolutionary algorithms: The mgbm criterion," in *2009 IEEE Congress on Evolutionary Computation*, 2009, p. 1263–1270.

[22] A. Arcuri and G. Fraser, *On Parameter Tuning in Search Based Software Engineering*, ser. Lecture Notes in Computer Science, 2011, vol. 6956, p. 33–47.

[23] P. Luong, D. Nguyen, S. Gupta, S. Rana, and S. Venkatesh, "Adaptive cost-aware bayesian optimization," *Knowledge-Based Systems*, vol. 232, p. 107481, 11 2021.

[24] T. Hildebrandt and J. Branke, "On using surrogates with genetic programming," *Evol. Comput.*, vol. 23, no. 3, pp. 343–367, 2015.

[25] J. H. Bussemaker, P. Saves, N. Bartoli, T. Lefebvre, and R. Lafage, "System architecture optimization strategies: dealing with expensive hierarchical problems," *Journal of Global Optimization*, vol. 91, no. 4, pp. 851–895, 2025.

[26] J. H. Bussemaker, P. Saves, N. Bartoli, T. Lefebvre, and B. Nagel, "Surrogate-based optimization of system architectures subject to hidden constraints," in *AIAA AVIATION FORUM AND ASCEND 2024*, 2024, p. 4401.

[27] J. H. Bussemaker, N. Bartoli, T. Lefebvre, P. D. Ciampa, and B. Nagel, "Effectiveness of surrogate-based optimization algorithms for system architecture optimization," in *AIAA Aviation 2021 forum*, 2021, p. 3095.

[28] S. Becker, H. Koziolek, and R. Reussner, "The palladio component model for model-driven performance prediction," *Journal of Systems and Software*, vol. 82, no. 1, p. 3–22, 2009.

[29] V. Titov, J. A. D. Pace, S. Frank, and A. van Hoorn, "Architecture optimization using surrogate-based incremental learning for quality-attribute analyses," in *22nd IEEE International Conference on Software Architecture, ICSA*, 2025, pp. 278–288.

[30] V. Cortellessa, D. Di Pompeo, V. Stoico, and M. Tucci, "Many-objective optimization of non-functional attributes based on refactoring of software models," *Inf. Softw. Technol.*, vol. 157, p. 107159, 2023.

[31] H. Xu and C. Jian, "A meta reinforcement learning-based virtual machine placement algorithm in mobile

edge computing," *Cluster Computing*, 2023.

[32] A. Beloglazov, J. Abawajy, and R. Buyya, "Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing," *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012, special Section: Energy efficiency in large-scale distributed systems.

[33] X. Fan, W.-D. Weber, and L. A. Barroso, "Power provisioning for a warehouse-sized computer," in *Proceedings of the 34th Annual International Symposium on Computer Architecture*, ser. ISCA '07, 2007, p. 13–23.

[34] V. Cortellessa, H. Singh, and B. Cukic, "Early reliability assessment of uml based software models," in *Proceedings of the 3rd international workshop on Software and performance*, 2002, p. 302–309.

[35] C. Li, T. Altamimi, M. H. Zargari, G. Casale, and D. C. Petriu, "Tulsa: A tool for transforming UML to layered queueing networks for performance analysis of data intensive applications," in *Quantitative Evaluation of Systems - 14th International Conference, QEST 2017, Berlin, Germany, September 5-7, 2017, Proceedings*, ser. Lecture Notes in Computer Science, N. Bertrand and L. Bortolussi, Eds., vol. 10503, 2017, pp. 295–299.

[36] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*, third edition draft ed., 2024.

[37] S. I. Wang and C. D. Manning, "Baselines and bigrams: Simple, good sentiment and topic classification," in *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2012, pp. 90–94.

[38] C. S. Leslie, E. Eskin, and W. S. Noble, "The spectrum kernel: A string kernel for svm protein classification," *Pacific Symposium on Biocomputing. Pacific Symposium on Biocomputing*, pp. 564–75, 2001.

[39] H. Borchani, G. Varando, C. Bielza, and P. Larranaga, "A survey on multi-output regression," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 5, no. 5, pp. 216–233, 2015.

[40] Y. Cao, B. J. Smucker, and T. J. Robinson, "On using the hypervolume indicator to compare pareto fronts: Applications to multi-criteria optimal experimental design," *Journal of Statistical Planning and Inference*, vol. 160, p. 60–74, 2015.

[41] N. Beume, B. Naujoks, and M. Emmerich, "Sms-emoa: Multiobjective selection based on dominated hypervolume," *European Journal of Operational Research*, vol. 181, no. 3, p. 1653–1669, 2007.

[42] H. Ishibuchi, H. Masuda, Y. Tanigaki, and Y. Nojima, "Modified distance calculation in generational distance and inverted generational distance," in *Evolutionary Multi-Criterion Optimization*, A. Gaspar-Cunha, C. Henggeler Antunes, and C. C. Coello, Eds., 2015, p. 110–125.

[43] M. Li and X. Yao, "Quality evaluation of solution sets in multiobjective optimisation: A survey," *ACM Computing Surveys*, vol. 52, no. 2, p. 1–38, 2020.

[44] S. Ö. Arik and T. Pfister, "Tabnet: Attentive interpretable tabular learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 8, 2021, pp. 6679–6687.